

Указатели и указателна аритметика

Ако T е тип данни, то T^* е указател към елемент от тип T .

Стойността на указател е адрес в паметта (уяко число) или `nullptr`, което е специална стойност, която означава не сочи никъде. Поневиде указателите са адреси, техният размер е размера на машинната дума, т.е. 4 байта при 32-битова архитектура и 8 байта при 64-битова архитектура.

```
int* ptr1; // неинициализиран
```

```
double* ptr2 = nullptr; // сочи никъде
```

```
int number = 10;
```

```
int* np = &number // сочи към number
```

```
int** npp = &np // сочи към np
```

$\&value$ връща адреса на променлива (резултатът е `value`)

$*pointer$ връща мястото в паметта сочено от указателя (резултатът е `value`)

Двете операции са дуални, т.е. $\&(*p) \Leftrightarrow p$ и $*(\&x) \Leftrightarrow x$

Указателната аритметика ни позволява по даден указател (отправна точка) да стигаме други клетки от паметта. Поневиде всеки тип има различен размер, големината на стъпката зависи от типа. Ако имаме T^*p , то $p+i$ прескача $i * \text{sizeof}(T)$ байта напред. Например:

$p+2$ при `char* p` прескача $2 \cdot 1 = 2$ байта напред

$p+2$ при `int* p` прескача $2 \cdot 4 = 8$ байта напред (при 32-битов `int`)

$p+2$ при `double* p` прескача $2 \cdot 8 = 16$ байта напред (при 64-битов `double`)

операцията `sizeof`

В C++ името на масив е указател към първия му елемент, който се държи подобно на $T^* \text{const}$, но не е възможно да се промени. Също така `arr[i] \Leftrightarrow *(arr+i)`

Едномерни масиви

Масивът е крайна редица от елементи от един и същ тип с произволен достъп за $O(1)$ до всеки един от тях. Елементите са разположени последователно в непрекъснат блок памет.

```
int arr[10]; // заделен масив с 10 елемента int
```

```
double arrayOfDoubles[3] = {0.5, 1.5, 2.0} // заделен масив с 3 елемента double и ги инициализира с дадените стойности
```

Възможно е да се каже, че в C++ няма проверки за коректност на индекса и достъп до индекс, който е извън масива е *undefined behavior*. Когато се подава масив като аргумент на функция реално се подава указател към първия елемент на масива и промени, които функцията прави се отразяват върху масива, т.е.

```
int main() {                                void modify(int arr[]) {  
    int arr[3] = {1, 2, 3};                  arr[0] = 3;  
    modify(arr);                             }  
    std::cout << arr[0];  
}
```

В този пример `std::cout` ще изпрати 3

Многомерни масиви

Многомерен масив е масив, който елементи са масиви

```
int m[2][3] = {{1, 2, 3}, {4, 5, 6}}
```

Физическото представяне в този случай е подобно на едномерния, елементите на един ред са последователни, а редовете също са един след друг (*)

Затова $arr[i][j] = *(*(arr+i)+j)$

(*) Това обаче не важи в следния случай, тук в другия случай редовете

```
int** arr = new int*[2];                ще са разпределени из heap-а
```

```
for (int i=0; i<2; ++i)
```

```
    arr[i] = new int[5];
```

Търсене в масив

• Линеенно търсене

```
int linearSearch(int arr[], int n, int x) {  
    for (int i = 0; i < n; ++i) {  
        if (arr[i] == x) {  
            return i;  
        }  
    }  
    return -1;  
}
```

В горния пример всеки елемент се проверява 1 по 1 докато не се намери желания елемент. Ако няма елемент със стойност x се връща -1 .

Времева сложност е $O(n)$

• Binary search

Можем да търсим в масив с алгоритъм с времева сложност $O(\log n)$, но е задължително той да бъде сортиран. В нашия пример

```
int binarySearch(int arr[], int n, int x) {
```

```
    int left = 0;
```

```
    int right = n - 1;
```

```
    while (left <= right) {
```

```
        int mid = left + (right - left) / 2;
```

```
        if (arr[mid] == x) {
```

```
            return mid;
```

```
        } else if (arr[mid] > x) {
```

```
            left = mid + 1;
```

```
        } else {
```

```
            // arr[mid] < x
```

```
            right = mid - 1;
```

```
        }
```

```
    }
```

```
    return -1;
```

ще приемем, че
arr е сортиран
във възходящ ред

• Сортиране (xx)

```
void selectionSort(int arr[], int n) {  
    for (int i=0; i<n; ++i) {  
        int minIndex=i;  
        for (int j=i+1; j<n; ++j) {  
            if (arr[j]<arr[minIndex]) {  
                minIndex=j;  
            }  
        }  
        std::swap(arr[i], arr[minIndex]);  
    }  
}
```

Вреговата сложност на
selectionSort е $O(n^2)$, но
не е стабилна. Пример
 $[5a, 5b, 2] \rightarrow [2, 5b, 5a]$

Символни низове

В C++ низът се представя като масив от char, в който последният символ
е $\backslash 0$

```
char word1[] = "Hello"; // Тук компилаторът добавя  $\backslash 0$ 
```

```
char word2[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Основни операции

- `std::cin >> s` - взема го разделител, но НЕ следи дали масивът е свършил, т.е.
ако имаме char масив с размер n ако запишем повече от n символа
е undefined behavior
- `std::cout << s` - взема докато не срещне $\backslash 0$. Ако няма $\backslash 0$ накрая отново
ни излезло извън масива и е undefined behavior
- `s[i]` - както при нормален масив

Има базисни функции в `<string>`

`strlen(s)` - броя на знаците на s без $\backslash 0$

`strcpy(dest, src)` - копира src в dest

`strcat(dest, src)` - конкатенира src след dest

Важно е да се каже, че strcpy и strcat нямат горн в dest да няма достатъчно заделено място, а strlen земе го, 10' и отново за масив без 10' имаме undefined behavior

```
char bad[] = {'H', 'e', 'l', 'l', 'o'};
```

```
std::cout << strlen(bad); // undefined behavior имаме 10' накрая
```

```
char s1[] = {"Hello"};
```

```
char s2[] = {"World"};
```

```
strcat(s1, s2); // undefined behavior няма достатъчно заделено място в s1
```

```
char s3[] = {"Hello World"};
```

```
strcpy(s1, s3); // undefined behavior няма достатъчно заделено място в s1
```

Рекурсията е метод за решаване на задачи при които задачата се свежда към една или няколко по-прости задачи, подобни на оригиналната.

Всяко рекурсивно решение има две части

- Основно - решение на най-простите случаи при което няма рекурсивно извикване

- Стъпка - свеждане на по-сложна задача към една или няколко по-прости

Ако рекурсията няма основно то тя е дълга безкрайна.

Пример

```
unsigned int fact(int n) {
```

```
    if (n == 0) {                // основно
```

```
        return 1;
```

```
}
```

```
    return n * fact(n-1); // стъпка
```

Пряка рекурсия имаме когато функцията извиква директно себе си както в горния пример

Косвена рекурсия имаме когато функцията извиква себе си не през други функция

Пример:

```
bool isOdd(int n);
```

```
bool isEven(int n) {
```

```
    if (n == 0) {
```

```
        return true;
```

```
    }
```

```
    return isOdd(n-1);
```

```
}
```

```
bool isOdd(int n) {
```

```
    if (n == 0) {
```

```
        return false;
```

```
    }
```

```
    return isEven(n-1);
```

```
}
```

Линейна рекурсия имае когато всяко извикване поразбуди най-много едно рекурсивно извикване. Например fact функцията по-горе

Дърбовична рекурсия имае когато рекурсивното извикване пороги повече рекурсивни извиквания. Например най-важната сложност на Fib:

```
int fib(int n) {
```

```
    if (n < 2) {
```

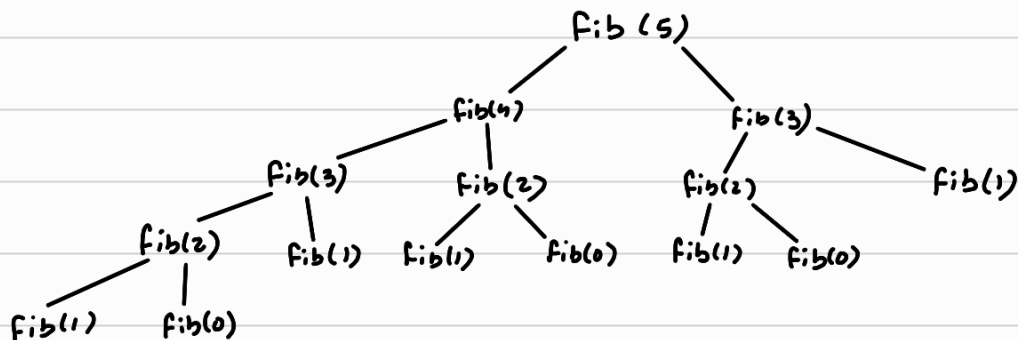
```
        return n;
```

```
    }
```

```
    return fib(n-1) + fib(n-2);
```

```
}
```

Например при fib(5) имаме:



Докумо при fact(5) имаме

fact(5) — fact(4) — fact(3) — fact(2) — fact(1) — fact(0)

(**)

```
void bubbleSort (int arr[], int n) {
```

```
    for (int i=0; i<n-1; ++i) {
```

```
        for (int j=0; j<n-1; ++j) {
```

```
            if (arr[j] > arr[j+1]) {
```

```
                swap(arr[j], arr[j+1]);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

bubbleSort е *смадушен*

Времетрае *сложност* на bubbleSort

е $O(n^2)$